

รายงาน

การวิเคราะห์และเตรียมข้อมูลสำหรับการวิเคราะห์ข้อมูลจากฐานข้อมูล Scopus

ทำโดย

นายพศินันท์ ประเสริฐการ	66011212032
นายอภิชาติ ศรียุทธ	66011212060
นายธีรวัฒน์ ไชยปัญญา	66011212104
นายธณัฐ กัญญาเยี่ยม	66011212174
นายธีรภัทร ราชดุขฎี	66011212180

เสนอ

ผศ.ดร.โอฬาริก สุรินทร์ตะ

รายงานการวิเคราะห์และเตรียมข้อมูลสำหรับการวิเคราะห์ข้อมูลจากฐานข้อมูล Scopus

รายงานฉบับนี้จัดทำขึ้นเพื่อวิเคราะห์และเตรียมข้อมูลจากฐานข้อมูล Scopus โดยใช้กระบวนการวิเคราะห์ข้อมูลเชิงสำรวจ (Exploratory Data Analysis: EDA) เพื่อทำความเข้าใจลักษณะและรูปแบบของข้อมูล รวมถึงการนำเสนอข้อมูลด้วย Visualization เพื่อให้เห็นข้อมูลและความสัมพันธ์ต่าง ๆ ได้ชัดเจนมากยิ่งขึ้น ทั้งนี้ ข้อมูลที่ผ่านการวิเคราะห์และเตรียมแล้วจะสามารถนำไปใช้ต่อในขั้นตอนการวิเคราะห์ข้อมูลในลำดับถัดไปได้

1. การสำรวจข้อมูลเบื้องต้น

1.1 ภาพรวมของ Dataset

ข้อมูลที่ใช้ในงานนี้มาจากการ Export บทความวิชาการจากฐานข้อมูล Scopus จำนวน 10 ไฟล์ CSV โดยแต่ละไฟล์เป็นข้อมูลของบทความในสาขาด้านคอมพิวเตอร์ที่แตกต่างกัน ดังนี้

ตารางที่ 1 ตารางฐานข้อมูล Scopus จำนวน 10 ไฟล์

ลำดับ	ชื่อไฟล์	สาขาวิชา (Subject)	จำนวนบทความ
1	scopus_export_Software.csv	Software	600
2	scopus_export_InformationSystems.csv	Information Systems	600
3	scopus_export_HumanComInteraction.csv	Human-Computer Interaction	600
4	scopus_export_Hardware.csv	Hardware	600
5	scopus_export_ComVision.csv	Computer Vision	600
6	scopus_export_ComScienceApp.csv	Computer Science Applications	600
7	scopus_export_ComNetworks.csv	Computer Networks	600
8	scopus_export_ComGraphicsDesign.csv	Computer Graphics & Design	600
9	scopus_export_ComMath.csv	Computational Mathematics	600
10	scopus_export_Ai.csv	Artificial Intelligence	600

นำไฟล์ทั้ง 10 ไฟล์เข้า Google Colab และอ่านข้อมูลด้วย `pd.read_csv()` โดยมีการรองรับกรณีที่ไฟล์ไม่ได้ใช้ encoding แบบ utf-8 ด้วยการเปลี่ยนไปใช้ latin1 จากนั้นรวมข้อมูลทั้งหมดด้วย `pd.concat()` และเพิ่มคอลัมน์ `Source_File` เพื่อเก็บชื่อไฟล์ต้นทางไว้สำหรับตรวจสอบข้อมูลภายหลัง ผลที่ได้คือข้อมูลทั้งหมด 6,000 แถว 23 คอลัมน์ และแต่ละไฟล์มีข้อมูล 600 แถวเท่ากัน

	File	Records
0	scopus_export_Ai.csv	600
1	scopus_export_ComGraphicsDesign.csv	600
2	scopus_export_ComMath.csv	600
3	scopus_export_ComNetworks.csv	600
4	scopus_export_ComScienceApp.csv	600
5	scopus_export_ComVision.csv	600
6	scopus_export_Hardware.csv	600
7	scopus_export_HumanComInteraction.csv	600
8	scopus_export_InformationSystems.csv	600
9	scopus_export_Software.csv	600

รูปที่ 1 ผลการรวมข้อมูลจากทั้ง 10 ไฟล์เป็น DataFrame เดียว ซึ่งมี 6,000 แถว 23 คอลัมน์ และตารางแสดงจำนวนบทความของแต่ละไฟล์ โดยทุกไฟล์มี 600 แถวเท่ากัน

ข้อมูลทั้ง 23 คอลัมน์เป็นข้อมูลบรรณานุกรมของบทความ เช่น Authors, Author full names, Author(s) ID, Title, Year, Source title, Volume, Issue, Art. No., Page start, Page end, Cited by, DOI, Link, Abstract, Author Keywords, Index Keywords, Document Type, Publication Stage, Open Access, Source, EID และ Source_File โดยคอลัมน์ที่เกี่ยวข้องกับการทำ Text Embedding โดยตรงคือ Title, Abstract, Author Keywords และ Index Keywords ส่วน EID ใช้เป็นรหัสอ้างอิงของบทความและใช้ตรวจสอบข้อมูลซ้ำ

1.2 การตรวจสอบโครงสร้างข้อมูล

จากการตรวจสอบด้วย df.shape และ df.dtypes พบว่าชุดข้อมูลมีขนาด 6,000 แถว 23 คอลัมน์ โดย Year และ Cited by เป็นข้อมูลประเภทตัวเลข(int64) ส่วนอีก 21 คอลัมน์เป็น object หรือข้อมูลข้อความ การตรวจสอบนี้ช่วยให้เห็นลักษณะของข้อมูลตั้งแต่ต้นและพบว่างานส่วนใหญ่จะเกี่ยวข้องกับการจัดการข้อมูลข้อความ เช่น การล้างช่องว่าง การจัดการค่าที่หายไป และการใช้ Regular Expression เนื่องจากข้อมูลส่วนใหญ่เป็นข้อความ การเตรียมข้อมูลจึงเน้นที่การทำความสะอาดข้อความก่อนนำไปสร้าง Text Embedding ในขั้นตอนต่อไป

1.3 การตรวจสอบ Missing Values

ตรวจสอบค่าที่หายไปด้วย df.isnull().sum() และคำนวณเป็นเปอร์เซ็นต์ของข้อมูลแต่ละคอลัมน์ ได้ผลดังนี้

ตารางที่ 2 ตารางแสดงจำนวนและเปอร์เซ็นต์ของ Missing Values ในแต่ละคอลัมน์

คอลัมน์	จำนวน Missing	%
Volume	11	0.18%
Issue	1427	23.78%
Art. No.	1555	25.92%
Page start	4467	74.45%
Page end	4470	74.50%
Author Keywords	489	8.15%
Index Keywords	798	13.30%
Open Access	6	0.10%
คอลัมน์อื่น ๆ (Authors, Title, Year, Abstract, DOI, EID ฯลฯ)	0	0.00%

จากผลการตรวจสอบ Page start และ Page end มีค่าหายไปมากที่สุด ประมาณ 74% รองลงมาคือ Art. No. และ Issue อย่างไรก็ตาม คอลัมน์เหล่านี้เป็นข้อมูลด้าน metadata ของการตีพิมพ์และไม่ได้ถูกนำไปใช้สร้าง Text Embedding จึงไม่ได้ส่งผลกระทบต่อขั้นตอนหลักของงาน สำหรับข้อมูลที่ใช้สร้าง Embedding โดยตรง Title และ Abstract ไม่มีค่าหายไป ส่วน Author Keywords และ Index Keywords มีค่าหายไปบางส่วน ซึ่งสามารถจัดการได้ด้วยการเติมค่าว่างก่อนนำข้อความไปต่อกัน

1.4 การตรวจสอบข้อมูลซ้ำ

ตรวจสอบข้อมูลซ้ำ 2 ส่วน ได้แก่

- ตรวจสอบแถวที่มีข้อมูลซ้ำกันทุกคอลัมน์ด้วย `df.duplicated().sum()` พบ 0 แถว
- ตรวจสอบ EID ที่ซ้ำกันด้วย `df['EID'].duplicated().sum()` พบ 295 รายการ

```
1 print("จำนวนแถวซ้ำทั้งหมด:",
2      df.duplicated().sum())
3
4 if 'EID' in df.columns:
5
6     print(
7         "EID ซ้ำ:",
8         df['EID'].duplicated().sum()
9     )
```

... จำนวนแถวซ้ำทั้งหมด: 0
EID ซ้ำ: 295

รูปที่ 2 ผลการตรวจสอบแถวที่ซ้ำกันทุกคอลัมน์และจำนวน EID ที่ซ้ำกัน

แม้จะไม่พบแถวที่มีข้อมูลเหมือนกันทุกคอลัมน์ แต่พบ EID ซ้ำกัน 295 รายการ ซึ่งหมายความว่าบทความบางส่วนปรากฏมากกว่าหนึ่งครั้งในข้อมูล เมื่อตรวจสอบต่อพบว่าบางบทความอยู่ในหลายสาขาวิชา จึงต้องจัดการข้อมูลส่วนนี้ก่อนนำไปใช้กับโมเดล เพราะบทความเดียวกันอาจมี Label มากกว่าหนึ่งค่า

1.5 ผลที่ได้จาก EDA

จากการสำรวจข้อมูลเบื้องต้น พบประเด็นสำคัญดังนี้

- ข้อมูลเริ่มต้นมี 6,000 แถว 23 คอลัมน์ และส่วนใหญ่เป็นข้อมูลประเภทข้อความ
- Title และ Abstract ไม่มี Missing Values ส่วน Author Keywords และ Index Keywords มี Missing Values บางส่วน
- คอลัมน์ด้าน metadata เช่น Page start, Page end, Art. No. และ Issue มี Missing Values ค่อนข้างสูง แต่ไม่ได้ถูกนำไปใช้สร้าง Text Embedding
- ไม่พบแถวที่ซ้ำกันทุกคอลัมน์ แต่พบ EID ซ้ำ 295 รายการ และบางรายการมี Subject มากกว่าหนึ่งค่า

จากผลดังกล่าวจึงนำข้อมูลเข้าสู่ขั้นตอน Data Cleaning เพื่อจัดการข้อมูลซ้ำและเตรียมข้อมูลให้เหมาะกับการนำไปสร้าง Embedding และใช้เป็นข้อมูลสำหรับ Machine Learning

1.6 การทำ Data Cleaning

หลังจากตรวจสอบข้อมูลในขั้นตอน EDA แล้ว จึงทำความสะอาดข้อมูลทั้งหมด 6 ขั้นตอน โดยจำนวนข้อมูลก่อนและหลังแต่ละขั้นตอนเป็นดังนี้

ตารางที่ 3 สรุปจำนวนข้อมูลก่อนและหลังการทำ Data Cleaning

ขั้นตอน	จำนวนก่อน	จำนวนหลัง	จำนวนที่ลดลง
ข้อมูลตั้งต้นหลังรวม 10 ไฟล์	–	6000	–
ลบบทความที่มี Subject มากกว่า 1 ค่า	6000	5424	576
ลบบทความที่ Title ซ้ำกัน	5424	5422	2
กรองเฉพาะ Title ที่เป็นภาษาอังกฤษ	5422	5342	80
รวมทั้งหมด	6000	5342	658

ขั้นตอนที่ 1 – สร้างคอลัมน์ Subject (Label)

สร้างคอลัมน์ Subject จากชื่อไฟล์ใน Source_File โดยตัดนามสกุล .csv ออก เช่น scopus_export_Software.csv จะได้ scopus_export_Software จากนั้นใช้คอลัมน์นี้เป็น Label สำหรับงาน Classification ในขั้นตอน Machine Learning

ขั้นตอนที่ 2 – ตรวจสอบและลบบทความที่มีมากกว่า 1 Subject

จากที่พบ EID ซ้ำในขั้น EDA จึงตรวจสอบต่อว่า EID เดียวกันมี Subject มากกว่าหนึ่งค่าหรือไม่ โดยใช้ `groupby("EID")["Subject"].nunique()` ผลการตรวจสอบพบว่ามีความ 281 บทความที่ EID เดียวกันแต่มี Subject มากกว่าหนึ่งค่า กรณีนี้ทำให้บทความเดียวกันมี Label มากกว่าหนึ่งค่า ซึ่งไม่เหมาะกับการ Classification จึงเลือกตัดบทความกลุ่มนี้ออกทั้งหมดแทนการเลือกเก็บ Subject ใด Subject หนึ่ง หลังจากลบแล้ว จำนวนข้อมูลลดลงจาก 6,000 เหลือ 5,424 แถว หรือลดลง 576 แถว

... จำนวนบทความที่พบมากกว่า 1 Subject: 281

	EID	Title	Subject
5065	2-s2.0-105003752945	A Contemporary Algebraic Attributes of m-Polar...	scopus_export_ComMath
3305	2-s2.0-105003752945	A Contemporary Algebraic Attributes of m-Polar...	scopus_export_ComScienceApp
2625	2-s2.0-105004697858	Gender Classification from Pashto Handwritten ...	scopus_export_ComVision
3584	2-s2.0-105004697858	Gender Classification from Pashto Handwritten ...	scopus_export_ComScienceApp
4214	2-s2.0-105014549301	Bringing Attention to CAD: Boundary Representa...	scopus_export_ComGraphicsDesign
2424	2-s2.0-105014549301	Bringing Attention to CAD: Boundary Representa...	scopus_export_ComVision
1949	2-s2.0-105014601110	Integrating artificial intelligence and quantu...	scopus_export_Hardware
5582	2-s2.0-105014601110	Integrating artificial intelligence and quantu...	scopus_export_Ai
3569	2-s2.0-105022205159	Classification and Analysis of Packaging Desig...	scopus_export_ComScienceApp
2728	2-s2.0-105022205159	Classification and Analysis of Packaging Desig...	scopus_export_ComVision
4675	2-s2.0-105024411506	A Stack-Free Parallel h-Adaptation Algorithm f...	scopus_export_ComGraphicsDesign

รูปที่ 3 ตัวอย่างบทความที่มี Subject มากกว่าหนึ่งค่า และผลการเปรียบเทียบจำนวนข้อมูลก่อนและหลังการลบ
ขั้นตอนที่ 3 – ตรวจสอบและลบ Title ที่ซ้ำกัน

หลังจากจัดการข้อมูลในคอลัมน์ Title ด้วย fillna และ strip() แล้ว พบว่ามี Title ซ้ำกัน 4 แถว หรือ 2 คู่ โดยแต่ละคู่มี EID ต่างกันและอยู่ในสาขา Hardware จึงลบข้อมูลที่มี Title ซ้ำกันและเก็บไว้เพียง 1 แถวต่อ Title เพื่อป้องกันไม่ให้ข้อมูลที่มีเนื้อหาเดียวกันกระจายไปอยู่ทั้งในชุด Train และ Test ซึ่งอาจทำให้เกิด Data Leakage และทำให้ผลการประเมินโมเดลสูงกว่าความเป็นจริง หลังจากขั้นตอนนี้เหลือข้อมูล 5,422 แถว

... จำนวนแถวที่ Title ซ้ำ: 4

	Title	Subject	EID
2032	ACALSim: A Scalable Parallel Simulation Framew...	scopus_export_Hardware	2-s2.0-105046362882
2369	ACALSim: A Scalable Parallel Simulation Framew...	scopus_export_Hardware	2-s2.0-105041128573
1932	Optimizing Dropout in LLM Training: Performanc...	scopus_export_Hardware	2-s2.0-105045966430
2399	Optimizing Dropout in LLM Training: Performanc...	scopus_export_Hardware	2-s2.0-105040931084

ก่อนลบ Title ซ้ำ: 5424
ลบออก: 2
เหลือ: 5422

รูปที่ 4 ตาราง Title ที่ซ้ำกัน 4 แถว พร้อม Subject และ EID และจำนวนข้อมูลก่อนและหลังการลบ
ขั้นตอนที่ 4 – จัดการ Missing Value ในคอลัมน์ Abstract

จาก EDA พบว่า Abstract ไม่มี Missing Value แต่ยังคงใช้ fillna("") และ strip() เพื่อป้องกันกรณีที่มีค่าที่หายไปหรือช่องว่างเกิดขึ้นในขั้นตอนอื่น และเพื่อให้ Abstract สามารถนำไปต่อกับข้อความส่วนอื่นได้โดยไม่มีปัญหา

ขั้นตอนที่ 5 – ตรวจจับและตัดข้อความ Copyright/License ออกจาก Abstract

ตรวจสอบ Abstract ว่ามีข้อความเกี่ยวกับลิขสิทธิ์หรือ License ปะปนอยู่หรือไม่ โดยค้นหาคำ เช่น copyright, ©, all rights reserved, creative commons, cc by และ license พบว่ามี 5,402 แถวที่ตรงกับเงื่อนไข ซึ่งส่วนใหญ่เป็นข้อความลิขสิทธิ์ที่ติดมากับข้อมูลจาก Scopus/Elsevier

เดิมการตัดข้อความใช้ `re.sub('copyright.*$', ...)` ซึ่งอาจมีปัญหาหากคำว่า copyright หรือ license ปรากฏอยู่ในเนื้อหาของบทความจริง เพราะข้อความหลังจากคำนั้นอาจถูกตัดออกไปด้วย จึงปรับวิธีให้ตรวจสอบเฉพาะช่วงท้ายของ Abstract ประมาณ 250 ตัวอักษร ซึ่งเป็นบริเวณที่พบข้อความลิขสิทธิ์ จากนั้นสุ่มตรวจผลลัพธ์ 10 แถวเพื่อดูว่าการตัดข้อความไม่กระทบเนื้อหาหลักของบทความ

	Title	Abstract
0	An LLM-based multi-agent framework for automat...	Due to the inherent variability of soil proper...
1	FERDOSI: fog-based energy-efficient reliable d...	This paper introduces FERDOSI (Fog-Based Energ...
2	LipidCruncher: an open-source platform for pro...	Background: Advances in mass spectrometry (MS)...
3	Making atomistic materials calculations access...	Despite the wide availability of density funct...
4	An empirical investigation of Large Language M...	Context: Automated compliance testing faces ch...
5	Optimized mutation scheduling for fuzzing base...	In the discovery of modern software vulnerabil...
6	The Harvard-Emory ECG Database	The Harvard-Emory ECG Database (HEEDB) is curr...
7	A Clinically Oriented Framework for Real-Time ...	Heart rate variability (HRV) is a well-establi...
8	STRIKER: a spectral metadata repairing tool fo...	The expansion of untargeted metabolomics has m...
9	pycol-vis: A Python package for image complexi...	Dataset complexity poses a significant challen...

รูปที่ 5 ตัวอย่างช่วงท้ายของ Abstract จำนวน 10 แถวหลังจัดการข้อความ Copyright/License เพื่อใช้ตรวจสอบผลลัพธ์

ขั้นตอนที่ 6 – ตรวจสอบและกรองภาษา (Language Filtering)

ใช้ไลบรารี `langdetect` ตรวจสอบภาษาของ Title โดยกำหนด `DetectorFactory.seed = 0` เพื่อให้ผลการตรวจสอบภาษาเหมือนเดิมเมื่อรันซ้ำ ผลการตรวจสอบภาษาของ Title ทั้ง 5,422 แถวมีดังนี้

ตารางที่ 4 ตารางแสดงจำนวน Title ที่ตรวจพบในแต่ละภาษา โดยข้อมูลส่วนใหญ่เป็นภาษาอังกฤษ

ภาษา	จำนวน
en (อังกฤษ)	5342
it (อิตาลี)	28
fr (ฝรั่งเศส)	10
ca (คาตาลัน)	8
de (เยอรมัน)	7
da (เดนมาร์ก)	6
pt (โปรตุเกส)	4
ro (โรมาเนีย)	4
es (สเปน)	3
อื่น ๆ (no, ru, tl, sv, ar, et, fi, af)	อย่างละ 1-2

พบ Title ที่ไม่ใช่ภาษาอังกฤษทั้งหมด 80 รายการ จึงกรองข้อมูลให้เหลือเฉพาะ Title ภาษาอังกฤษ เพื่อให้สอดคล้องกับโมเดล Embedding ที่จะใช้ในขั้นตอนต่อไป หลังจากกรองแล้วเหลือข้อมูล 5,342 แถว

สรุปผลรวมการทำความสะอาดข้อมูล

หลังจากผ่านทุกขั้นตอนของ Data Cleaning แล้ว ข้อมูลลดจาก 6,000 เหลือ 5,342 แถว หรือลดลงทั้งหมด 658 แถว คิดเป็นประมาณ 11% และจำนวนคอลัมน์เพิ่มจาก 23 เป็น 26 คอลัมน์ โดยคอลัมน์ที่เพิ่มขึ้นคือ Subject, Title_Language และ embedding_text นอกจากนี้ยังตรวจสอบ embedding_text ว่ามีแถวที่เป็นค่าว่างหรือไม่ และไม่พบข้อมูลที่ว่าง จึงไม่ต้องลบข้อมูลเพิ่มเติมในขั้นตอนนี้ จากนั้นบันทึกข้อมูลที่ทำสะอาดแล้วเป็น `scopus_cleaned.csv`

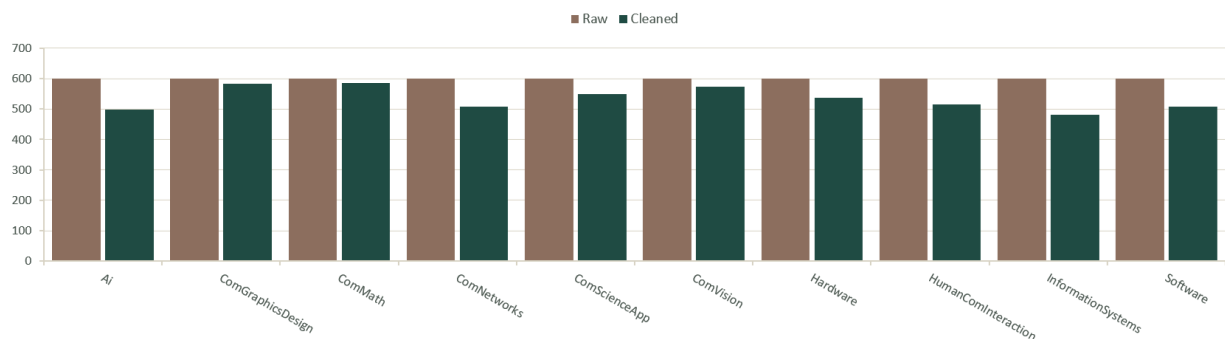
2. การทำ Visualization

การวิเคราะห์ข้อมูลเชิงสำรวจ (Exploratory Data Analysis: EDA) ถูกดำเนินการเพื่อศึกษาลักษณะโครงสร้าง และความสมบูรณ์ของชุดข้อมูลบทความวิจัยจาก Scopus จำนวน 10 สาขาวิชา โดยพิจารณาข้อมูลในคอลัมน์ Title, Abstract, Author Keywords และ Index Keywords ซึ่งเป็นข้อมูลที่น่าไปใช้ในกระบวนการสร้าง Text Embedding และการจำแนกสาขาวิชา Visualization แต่ละรูปแบบถูกสร้างขึ้นเพื่อแสดงคุณลักษณะที่แตกต่างกันของข้อมูล เช่น จำนวนบทความก่อนและหลังการทำความสะอาดข้อมูล ค่าที่หายไป ความยาวของข้อความ จำนวน Keywords และความถี่ของคำที่ปรากฏในแต่ละประเภทข้อมูล ผลจาก Visualization ช่วยให้เห็นความสัมพันธ์และรูปแบบของข้อมูล รวมถึงช่วยประเมินความพร้อมของข้อมูลก่อนนำไปใช้ในขั้นตอนการสร้าง Embedding และการพัฒนาโมเดล Machine Learning

2.1 จำนวนบทความต่อสาขาวิชา ก่อนและหลังการทำความสะอาดข้อมูล

Visualization นี้ใช้ Grouped Bar Chart เพื่อเปรียบเทียบจำนวนบทความในแต่ละสาขาวิชาก่อนและหลังการทำความสะอาดข้อมูล โดยข้อมูลดิบมีจำนวนบทความ 600 บทความต่อสาขา รวมทั้งหมด 10 สาขา และหลังจากดำเนินการ Data Cleaning เช่น การตัดบทความที่มีหลาย Subject ซ้ำ และการกรองบทความที่ไม่ใช่ภาษาอังกฤษ ทำให้จำนวนบทความในแต่ละสาขาลดลงแตกต่างกัน

Records per Subject — Raw vs Cleaned



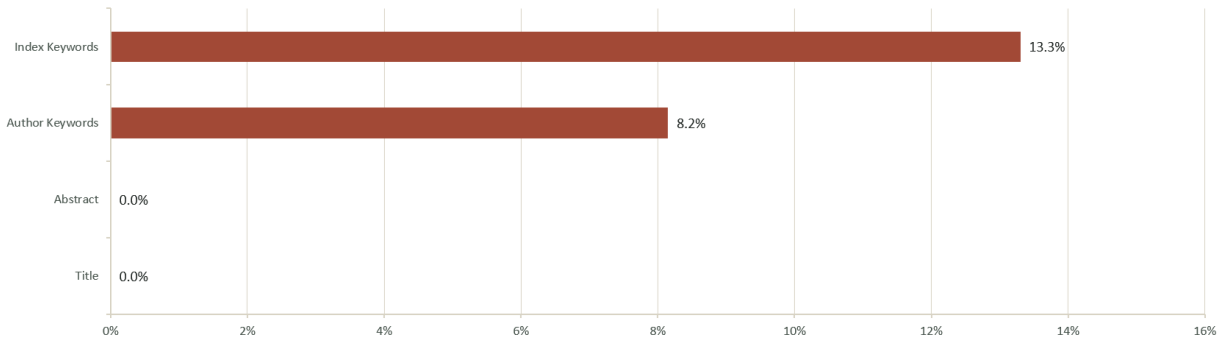
รูปที่ 6 จำนวนบทความต่อสาขาวิชา ก่อนและหลังการทำความสะอาดข้อมูล

2.2 ค่าที่หายไปของข้อมูล

Visualization นี้ใช้ Horizontal Bar Chart เพื่อแสดงสัดส่วนข้อมูลที่มีค่าหายไปในแต่ละคอลัมน์ ได้แก่ Title, Abstract, Author Keywords และ Index Keywords โดยพบว่า Title และ Abstract ไม่มีค่าหายไป ขณะที่ Author Keywords มีค่าหายไปประมาณ 8.15% และ Index Keywords มีค่าหายไปประมาณ 13.30%

ซึ่งเป็นคอลัมน์ที่มี Missing Values มากที่สุด ความสัมพันธ์ของข้อมูลแสดงให้เห็นถึง ความสมบูรณ์ของข้อมูลแต่ละประเภท และช่วยระบุคอลัมน์ที่จำเป็นต้องจัดการ Missing Values ก่อนนำข้อมูลไปสร้าง Text Embedding

Missing Values by Column (%)

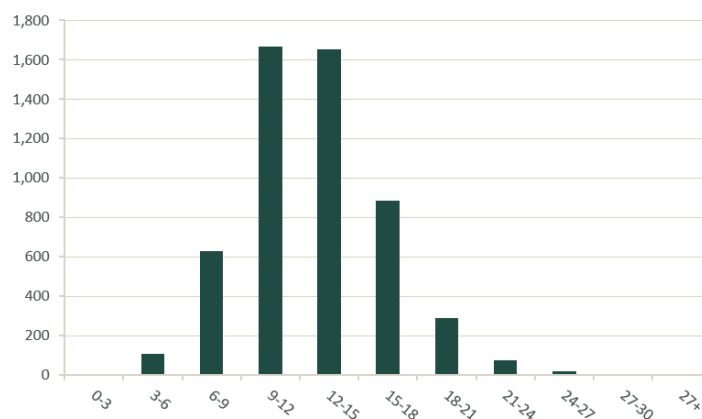


รูปที่ 7 ค่าที่หายไปของข้อมูล

2.3 การกระจายความยาวของ Title

Visualization นี้ใช้ Histogram เพื่อแสดงการกระจายจำนวนคำในชื่อบทความหลังการทำความสะอาดข้อมูลพบว่าชื่อบทความส่วนใหญ่อยู่ในช่วงประมาณ 9-15 คำ แสดงให้เห็นว่าชื่อบทความส่วนใหญ่มีความยาวไม่มาก และไม่มีแนวโน้มที่จะมีความยาวเกินข้อจำกัดของกระบวนการสร้าง Embedding ซึ่งกำหนด max_length=512 tokens ดังนั้นข้อมูลใน Title ส่วนใหญ่จึงสามารถนำไปใช้เป็นส่วนหนึ่งของข้อความสำหรับสร้าง Embedding ได้ โดยไม่เกิดการตัดข้อความอย่างมีนัยสำคัญ

Title Length Distribution (words)

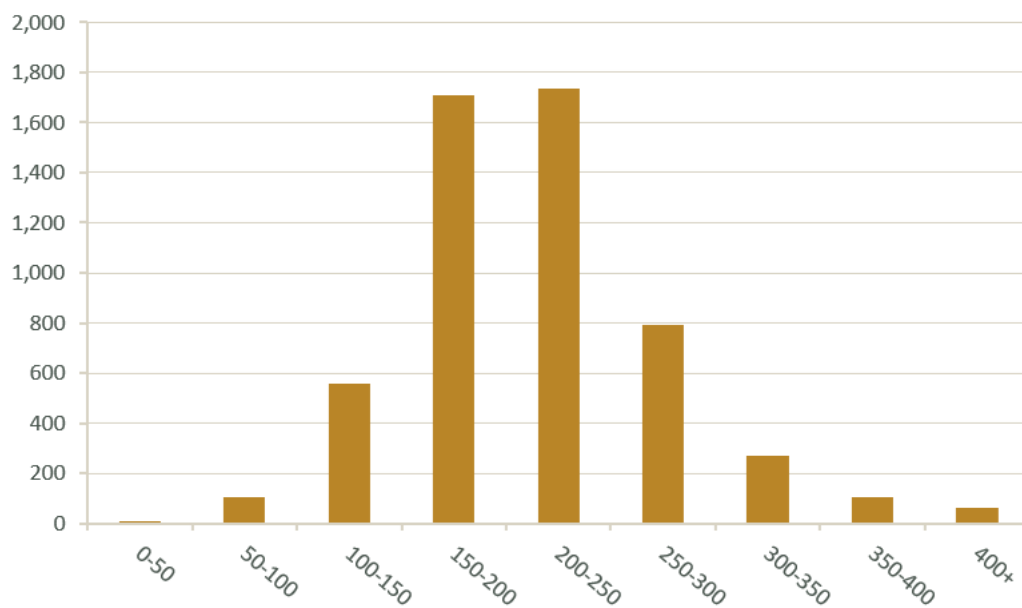


รูปที่ 8 การกระจายความยาวของ Title

2.4 การกระจายความยาวของ Abstract

Visualization นี้ใช้ Histogram เพื่อแสดงการกระจายจำนวนคำใน Abstract ของบทความหลังการทำความสะอาดข้อมูล พบว่าบทความส่วนใหญ่อยู่ในช่วงประมาณ 150–300 คำ ซึ่งมีความยาวมากกว่า Title อย่างชัดเจน ดังนั้น Abstract จึงมีส่วนสำคัญในการให้บริบทและรายละเอียดของเนื้อหาบทความ เมื่อนำไปรวมกับข้อมูลส่วนอื่นเพื่อสร้าง Embedding ความสัมพันธ์ที่เห็นจาก Visualization นี้คือ ความยาวของ Abstract กับปริมาณเนื้อหาที่ใช้เป็นบริบทสำหรับตัวแทนข้อมูลของบทความ

Abstract Length Distribution (words)

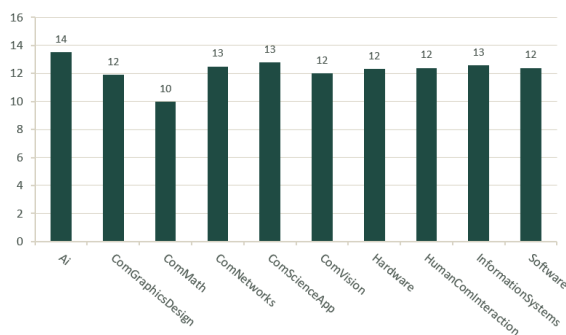


รูปที่ 9 การกระจายความยาวของ Abstract

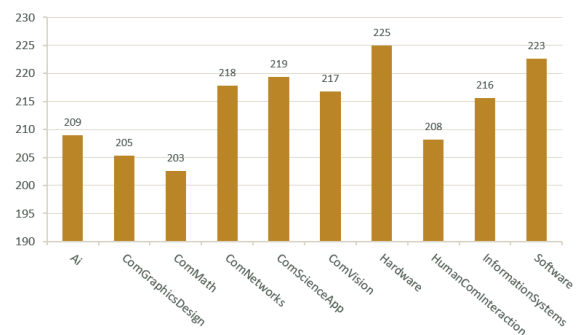
2.5 ความยาวเฉลี่ยของ Title และ Abstract แยกตามสาขาวิชา

Visualization นี้ใช้ Bar Chart เพื่อเปรียบเทียบความยาวเฉลี่ยของ Title และ Abstract ระหว่าง 10 สาขาวิชา โดยพบว่าความยาวของ Title ในแต่ละสาขามีค่าใกล้เคียงกันประมาณ 10–14 คำ ขณะที่ Abstract มีความยาวเฉลี่ยประมาณ 203–225 คำ โดย Hardware มีค่าเฉลี่ยความยาว Abstract สูงที่สุด และ ComMath มีค่าต่ำที่สุด ความสัมพันธ์ของข้อมูลแสดงให้เห็นว่า สาขาวิชามีความแตกต่างด้านความยาวของข้อความเพียงเล็กน้อย จึงไม่สามารถใช้ความยาวของ Title หรือ Abstract เพียงอย่างเดียวในการจำแนกสาขาวิชาได้อย่างชัดเจน

Avg. Title Length (words)



Avg. Abstract Length (words)

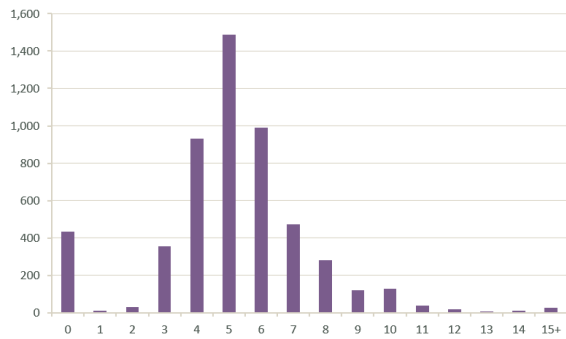


รูปที่ 10 ความยาวเฉลี่ยของ Title และ Abstract แยกตามสาขาวิชา

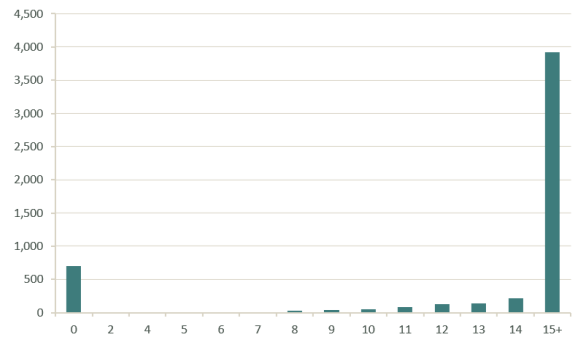
2.6 จำนวน Author Keywords และ Index Keywords ตอบบทความ

Visualization นี้ใช้ Histogram เพื่อศึกษาการกระจายจำนวน Keywords ของแต่ละบทความ โดย Author Keywords ซึ่งเป็นคำสำคัญที่ผู้เขียนกำหนดเอง มีจำนวนแตกต่างกันค่อนข้างมาก และมีบทความจำนวนหนึ่งที่ไม่มี Author Keywords ขณะที่ Index Keywords ซึ่งมาจากระบบจัดทำดัชนีของ Scopus มีจำนวนตอบบทความสูงกว่าอย่างชัดเจน โดยบทความส่วนใหญ่มี Index Keywords มากกว่า 15 คำ ความสัมพันธ์ของข้อมูลแสดงให้เห็นว่า Index Keywords มีปริมาณคำศัพท์ทางเทคนิคตอบบทความมากกว่า Author Keywords จึงอาจให้ข้อมูลสำหรับการวิเคราะห์เนื้อหาและการจำแนกสาขาวิชาได้มากกว่า

Author Keywords per Article



Index Keywords per Article

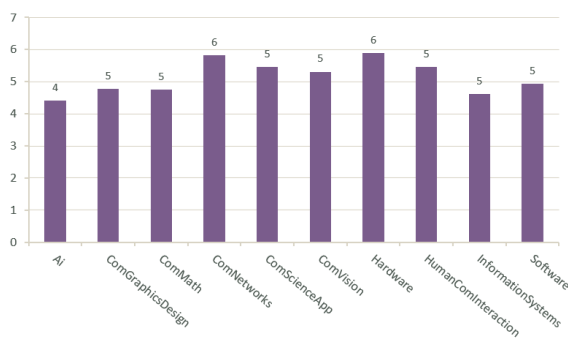


รูปที่ 11 จำนวน Author Keywords และ Index Keywords ต่อบทความ

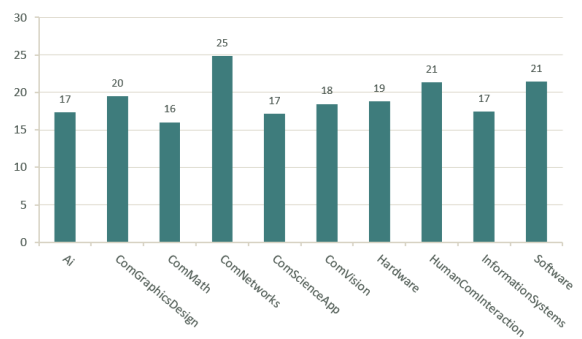
2.7 จำนวน Keywords เฉลี่ยแยกตามสาขาวิชา

Visualization นี้ใช้ Bar Chart เพื่อเปรียบเทียบจำนวน Author Keywords และ Index Keywords เฉลี่ยของแต่ละสาขาวิชา พบว่า Author Keywords มีความแตกต่างระหว่างสาขาไม่มากนัก โดยมีค่าเฉลี่ยประมาณ 4.4–5.9 คำต่อบทความ ในขณะที่ Index Keywords มีความแตกต่างมากกว่า โดย ComNetworks มีค่าเฉลี่ยสูงที่สุดประมาณ 24.89 คำ และ ComMath ต่ำที่สุดประมาณ 16.02 คำ ความสัมพันธ์ดังกล่าวแสดงให้เห็นว่า Index Keywords มีความแตกต่างระหว่างสาขาวิชามากกว่า Author Keywords จึงมีศักยภาพในการนำมาใช้เป็น Feature สำหรับการจำแนกสาขาวิชา

Avg. Author Keywords



Avg. Index Keywords

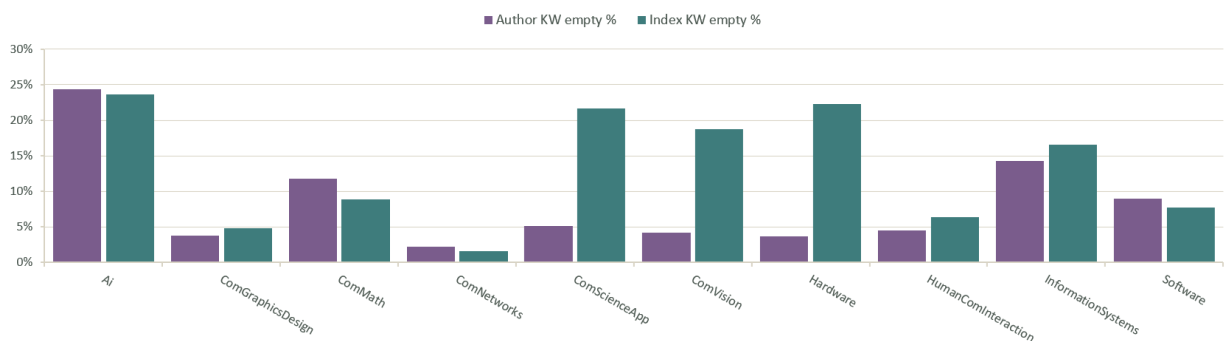


รูปที่ 12 จำนวน Keywords เฉลี่ยแยกตามสาขาวิชา

2.8 สัดส่วนบทความที่ไม่มี Keywords แยกตามสาขาวิชา

Visualization นี้ใช้ Bar Chart เพื่อเปรียบเทียบสัดส่วนบทความที่ไม่มี Author Keywords และ Index Keywords ในแต่ละสาขาวิชา พบว่าสาขา AI มีอัตราการไม่มี Keywords สูงเมื่อเปรียบเทียบกับหลายสาขา โดย Author Keywords ไม่มีข้อมูลประมาณ 24.4% และ Index Keywords ประมาณ 23.6% ในขณะที่บางสาขามีอัตราการไม่มี Keywords ต่ำกว่ามาก ความสัมพันธ์ของข้อมูลแสดงให้เห็นว่า ความครบถ้วนของ Keywords แตกต่างกันตามสาขาวิชา ซึ่งอาจส่งผลต่อประสิทธิภาพของโมเดล Classification โดยเฉพาะสาขาที่มีข้อมูล Keywords หายไปในสัดส่วนสูง

Empty Keyword Rate by Subject (%)

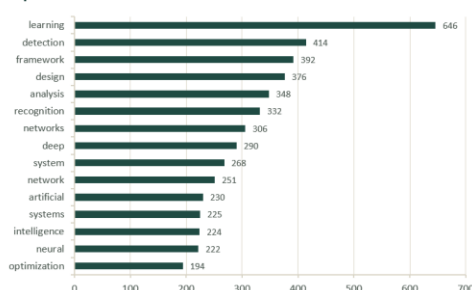


รูปที่ 13 สัดส่วนบทความที่ไม่มี Keywords แยกตามสาขาวิชา

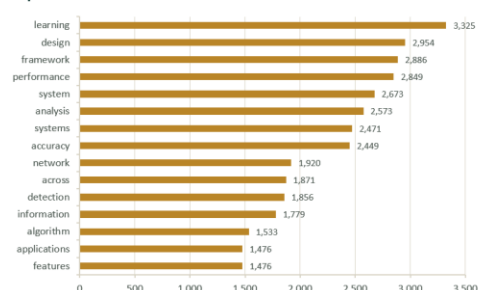
2.9 คำที่พบบ่อยที่สุดใน Title และ Abstract

Visualization นี้ใช้การแสดง Word Frequency เพื่อวิเคราะห์คำที่ปรากฏบ่อยที่สุดใน Title และ Abstract ของบทความทั้งหมด โดยใน Title คำว่า “learning” พบมากที่สุดจำนวน 646 ครั้ง ส่วนใน Abstract พบคำว่า “learning” จำนวน 3,325 ครั้ง นอกจากนี้ยังพบคำที่เกี่ยวข้องกับการทดลองและการประเมินผล เช่น performance และ accuracy ความสัมพันธ์ของข้อมูลแสดงให้เห็นถึง หัวข้อและแนวทางการวิจัยที่ปรากฏบ่อยในชุดข้อมูล และสะท้อนว่าข้อมูลมีแนวโน้มเกี่ยวข้องกับงานด้าน AI และ Machine Learning ในระดับหนึ่ง

Top 15 Words — Title



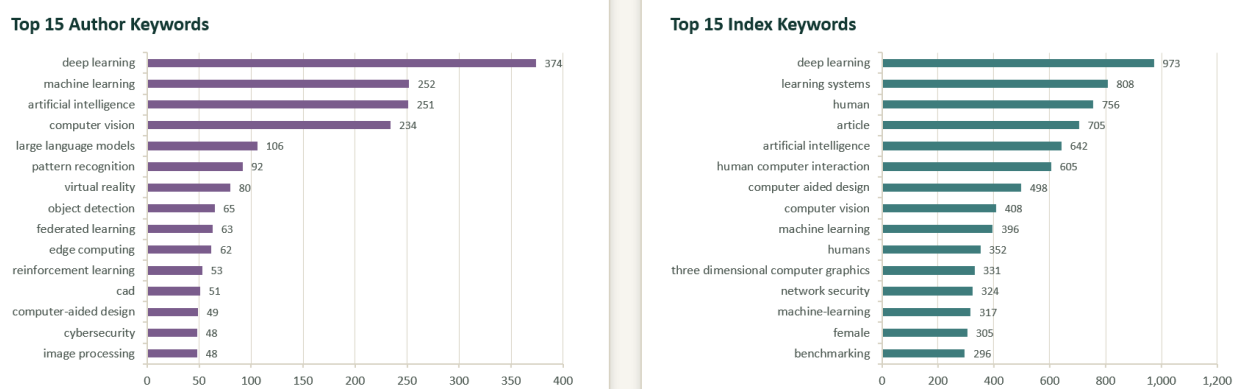
Top 15 Words — Abstract



รูปที่ 14 คำที่พบบ่อยที่สุดใน Title และ Abstract

2.10 คำสำคัญที่พบบ่อยที่สุดใน Author Keywords และ Index Keywords

Visualization นี้ใช้การวิเคราะห์ความถี่ของคำใน Author Keywords และ Index Keywords เพื่อแสดงคำสำคัญที่ปรากฏบ่อยที่สุดในชุดข้อมูล โดย Author Keywords พบคำว่า “deep learning” มากที่สุด จำนวน 374 ครั้ง สะท้อนหัวข้อที่ผู้เขียนให้ความสำคัญ ส่วน Index Keywords พบคำ เช่น “deep learning”, “human” และ “article” ในความถี่สูง อย่างไรก็ตาม คำบางส่วน เช่น human, article และ female มีลักษณะเป็น Metadata ที่ระบบ Scopus กำหนดโดยอัตโนมัติและอาจไม่ได้สะท้อนเนื้อหาทางวิชาการโดยตรง ดังนั้น Visualization นี้จึงช่วยแสดง ความแตกต่างระหว่างคำสำคัญที่ผู้เขียนกำหนดเองกับคำที่ระบบจัดทำดัชนี และช่วยในการพิจารณาว่า คำใดควรนำไปใช้เป็น Feature ในขั้นตอนต่อไป



รูปที่ 15 คำสำคัญที่พบบ่อยที่สุดใน Author Keywords และ Index Keywords

3. การเตรียมข้อมูลสำหรับ Text Embedding / Feature Vector

3.1 ข้อมูลที่เลือกใช้

หลังจากทำความสะอาดข้อมูลแล้ว เลือกเฉพาะคอลัมน์ที่เกี่ยวข้องกับการทำ Embedding และการอ้างอิงข้อมูลมาเก็บไว้ใน selected_df ได้แก่ EID, Title, Abstract, Author Keywords, Index Keywords, Subject และ embedding_text

- Title ชื่อบทความ ใช้เป็นข้อมูลหลักที่ช่วยบอกเนื้อหาและหัวข้อของงานวิจัย
- Abstract บทคัดย่อ ให้รายละเอียดของบทความมากกว่า Title และช่วยให้โมเดลเห็นบริบทของงานวิจัย
- Author Keywords คำสำคัญที่ผู้เขียนกำหนดขึ้นเอง เพื่อบอกประเด็นที่ต้องการเน้น
- Index Keywords คำสำคัญจากระบบจัดทำดัชนีของ Scopus ช่วยเพิ่มคำศัพท์ทางเทคนิคที่เกี่ยวข้องกับบทความ
- Subject Label ของบทความ ใช้เป็น Target สำหรับงาน Classification
- EID รหัสประจำบทความ ใช้สำหรับอ้างอิงและตรวจสอบข้อมูล ไม่ได้นำไปใช้เป็น Feature

	Title	Abstract	Author Keywords	Index Keywords	embedding_text
0	AI-agent-empowered edge-IoT framework for pers...	The integration of Artificial Intelligence (AI...	Anomaly detection; Artificial intelligence; Da...	AI-agent-empowered edge-IoT framework for pers...	
1	Toward a robust and generalizable metamaterial...	Advances in material functionalities drive inn...	Artificial intelligence; Inverse problems; Mat...	Toward a robust and generalizable metamaterial...	
2	Artificial intelligence assisted colorectal le...	Computer-aided colonoscopy (CAC) may improve p...	Oncology; Patient treatment; Screening; Cancer...	Artificial intelligence assisted colorectal le...	

รูปที่ 16 ข้อมูลที่เลือกใช้

ตัวอย่างข้อมูล 5 แถวแรกของ selected_df และคอลัมน์ที่เลือกใช้ในขั้นตอน Text Embedding

3.2 การเตรียมข้อมูล

ก่อนนำข้อมูลไปสร้างข้อความสำหรับ Embedding ทำความสะอาดคอลัมน์ Title, Abstract, Author Keywords และ Index Keywords ด้วย fillna(""), astype(str) และ strip() เพื่อให้ข้อมูลอยู่ในรูปแบบข้อความ และไม่มี NaN จากนั้นสร้างคอลัมน์ embedding_text โดยนำ Title, Abstract, Author Keywords และ Index Keywords มาต่อกันเป็นข้อความเดียว และใช้ [SEP] เป็นตัวคั่นระหว่างแต่ละส่วน เพื่อให้เห็นขอบเขตของข้อมูลแต่ละส่วนได้ชัดเจน

	Title	Abstract	Author Keywords	Index Keywords	embedding_text
0	AI-agent-empowered edge-IoT framework for pers...	The integration of Artificial Intelligence (AI...		Anomaly detection; Artificial intelligence; Da...	AI-agent-empowered edge-IoT framework for pers...
1	Toward a robust and generalizable metamaterial...	Advances in material functionalities drive inn...		Artificial intelligence; Inverse problems; Mat...	Toward a robust and generalizable metamaterial...
2	Artificial intelligence assisted colorectal le...	Computer-aided colonoscopy (CAC) may improve p...	Oncology; Patient treatment; Screening; Cancer...		Artificial intelligence assisted colorectal le...

รูปที่ 17 การเตรียมข้อมูล

ตัวอย่าง Title, Abstract, Author Keywords, Index Keywords และ embedding_text ที่สร้างจากข้อมูลทั้ง

4 ส่วน

3.3 การสร้าง Text Embedding

ใช้โมเดล SPECTER2 (allenai/specter2_base) สำหรับสร้าง Document Embedding ของบทความวิชาการ โดยการใช้งานไม่ได้โหลดเฉพาะ base encoder เท่านั้น แต่โหลด Adapter ของ SPECTER2 (proximity adapter) เพิ่มด้วย เพื่อให้ได้ Embedding ตามรูปแบบที่โมเดลออกแบบไว้

ขั้นตอนการสร้าง Embedding มีดังนี้

โหลด Tokenizer และโมเดลด้วย AutoTokenizer และ AutoAdapterModel จากไลบรารี adapters

ตรวจสอบอุปกรณ์ที่ใช้ประมวลผล โดยเลือก CUDA หากมี GPU และ CPU หากไม่มี GPU ในการรันครั้งนี้ใช้ GPU T4 บน Google Colab

ทดลองสร้าง Embedding จากข้อความ 1 แถวก่อน ได้ขนาด [1, 768]

สร้างฟังก์ชัน create_embeddings() เพื่อประมวลผลแบบ Batch ครั้งละ 16 ข้อความ และจำกัดความยาวไม่เกิน 512 token

นำค่า Embedding จาก token แรก ([CLS]) ของ last_hidden_state มาใช้เป็นตัวแทนของเอกสาร

สร้าง Embedding ให้ข้อมูลทั้งหมด 5,342 แถว ได้ผลลัพธ์ขนาด [5342, 768]

```
print("Embedding shape:")
print(embeddings.shape)
```

100%  334/334 [02:52<00:00, 2.01it/s]

Embedding shape:
torch.Size([5342, 768])

รูปที่ 18 ขนาดของ Embedding ที่ได้จากข้อมูลทั้งหมด โดยมี 5,342 แถว และแต่ละบทความมีเวกเตอร์ 768

มิติ

3.4 Feature Vector ที่ได้

ผลจากการสร้าง Embedding คือ Feature Vector ขนาด 768 มิติสำหรับบทความแต่ละรายการ รวมทั้งหมด 5,342 บทความ ก่อนนำไปใช้กับ Machine Learning มีการทำ L2-normalization กับเวกเตอร์แต่ละตัว เพื่อให้เวกเตอร์มีขนาดหน่วยเดียวกัน จากนั้นแปลงเป็น NumPy array ในตัวแปร X ซึ่งมีขนาด (5342, 768)

ดังนั้น Feature ที่ใช้ในโมเดลไม่ได้มาจาก EID หรือ Subject โดยตรง แต่เป็นตัวเลข 768 มิติที่ได้จากเนื้อหาของบทความผ่านโมเดล SPECTER2

3.5 การเตรียมข้อมูลสำหรับ Machine Learning

การแยก Feature และ Label

- Feature (X) คือ Embedding Vector ที่ผ่าน L2-normalization แล้ว มีขนาด (5342, 768)
- Label/Target (y) คือค่า Subject ของบทความจำนวน 5,342 ค่า โดยมีทั้งหมด 10 คลาสตาม 10 สาขาวิชา

การแบ่ง Train/Test

แบ่งข้อมูลด้วย train_test_split() ในสัดส่วน 80/20 และใช้ stratify=y เพื่อให้สัดส่วนของแต่ละ Subject ในชุด Train และ Test ใกล้เคียงกัน พร้อมกำหนด random_state=42 เพื่อให้สามารถทำซ้ำการแบ่งข้อมูลได้

ผลการแบ่งข้อมูลเป็น X_train ขนาด (4273, 768), X_test ขนาด (1069, 768), y_train ขนาด (4273,) และ y_test ขนาด (1069,)

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print("X_train:", X_train.shape)
print("X_test :", X_test.shape)
print("y_train:", y_train.shape)
print("y_test :", y_test.shape)

X_train: (4273, 768)
X_test : (1069, 768)
y_train: (4273,)
y_test : (1069,)
```

รูปที่ 19 ขนาดของข้อมูลในชุด Train และ Test หลังจากแบ่งข้อมูลแบบ Stratify

การเทรนและประเมินผล

นำ Feature Vector ไปใช้กับโมเดล Logistic Regression โดยกำหนด max_iter=1000 จากนั้นใช้โมเดลทำนายข้อมูลในชุด Test ได้ Accuracy โดยรวมเท่ากับ 0.6324 หรือประมาณ 63.24%

ตารางที่ 5 ตาราง ค่า Accuracy และ Classification Report ที่ได้จากการทดสอบโมเดล Logistic Regression

Subject	Precision	Recall	F1-score	Support
Ai	0.56	0.65	0.6	100
ComGraphicsDesign	0.79	0.84	0.81	117
ComMath	0.78	0.89	0.83	117
ComNetworks	0.67	0.69	0.68	101
ComScienceApp	0.45	0.22	0.29	110
ComVision	0.58	0.84	0.69	115
Hardware	0.67	0.7	0.68	108
HumanComInteraction	0.55	0.51	0.53	103
InformationSystems	0.41	0.26	0.32	96
Software	0.65	0.63	0.64	102
Macro avg	0.61	0.62	0.61	1069
Weighted avg	0.62	0.63	0.61	1069

เมื่อดูจาก F1-score รายสาขา พบว่า ComMath และ ComGraphicsDesign มีผลการทำนายดีที่สุด โดยมี F1-score เท่ากับ 0.83 และ 0.81 ตามลำดับ ส่วน ComScienceApp และ InformationSystems มี F1-score ต่ำที่สุดที่ 0.29 และ 0.32 ตามลำดับ อย่างไรก็ตาม จากข้อมูลในไฟล์ยังไม่มี การวิเคราะห์เพิ่มเติมว่าเหตุใดแต่ละสาขาจึงมีผลแตกต่างกัน จึงไม่สามารถระบุสาเหตุได้อย่างชัดเจนจากผลการทดลองนี้

การบันทึกผลลัพธ์

หลังจากสร้าง Embedding และเตรียมข้อมูลเสร็จแล้ว มีการบันทึกผลลัพธ์ไว้ 2 ไฟล์ ได้แก่ specter2_embeddings.npy สำหรับเก็บ Embedding ทั้งหมด และ scopus_cleaned_with_embedding_text.csv สำหรับเก็บข้อมูลที่มีคอลัมน์ embedding_text เพื่อให้สามารถนำผลลัพธ์ไปใช้ต่อได้โดยไม่ต้องสร้าง Embedding ใหม่ทุกครั้ง

4. สรุปกระบวนการทั้งหมด

กระบวนการทั้งหมดเริ่มจากการนำบทความวิชาการจาก Scopus จำนวน 10 สาขา รวม 6,000 บทความมารวมเป็น Dataset เดียว จากนั้นตรวจสอบข้อมูลด้วย EDA ทั้งด้านโครงสร้าง Missing Values และข้อมูลซ้ำ

หลังจากนั้นทำ Data Cleaning โดยสร้าง Subject จากชื่อไฟล์ ลบบทความที่มี EID เดียวกันแต่มีหลาย Subject ลบ Title ที่ซ้ำกัน ตัดข้อความ Copyright/License ที่ติดมากับ Abstract และกรอง Title ให้เหลือเฉพาะภาษาอังกฤษ ทำให้เหลือข้อมูล 5,342 แถว 26 คอลัมน์

เมื่อได้ข้อมูลที่ผ่านการทำความสะอาดแล้ว จึงนำ Title, Abstract, Author Keywords และ Index Keywords มารวมเป็น embedding_text โดยใช้ [SEP] เป็นตัวคั่น แล้วสร้าง Text Embedding ด้วย SPECTER2 พร้อม Proximity Adapter ได้ Feature Vector ขนาด 768 มิติต่อบทความ

จากนั้นทำ L2-normalization และนำ Feature Vector ไปแบ่งเป็นข้อมูล Train/Test ในสัดส่วน 80/20 ก่อนนำไปทดลองกับ Logistic Regression ผลการทดสอบได้ Accuracy 63.24%

สำหรับ Visualization ในไฟล์เดิมยังไม่มีการสร้างกราฟจริง ดังนั้นหากต้องการให้รายงานสมบูรณ์ขึ้น สามารถเพิ่มกราฟ เช่น จำนวนบทความแต่ละสาขา Missing Values จำนวนข้อมูลก่อนและหลัง Cleaning การกระจายภาษา Confusion Matrix และผล Precision/Recall/F1 ของแต่ละสาขาได้

โดยรวมแล้ว ขั้นตอนของงานครอบคลุมตั้งแต่การเตรียม Dataset การตรวจสอบและทำความสะอาดข้อมูล การสร้างข้อความสำหรับ Embedding การสร้าง Feature Vector ไปจนถึงการนำ Feature ไปทดลองกับ Machine Learning โดยผลลัพธ์ที่ได้สามารถนำไปใช้เป็นข้อมูลตั้งต้นสำหรับการทดลองหรือพัฒนาขั้นตอนต่อไปได้